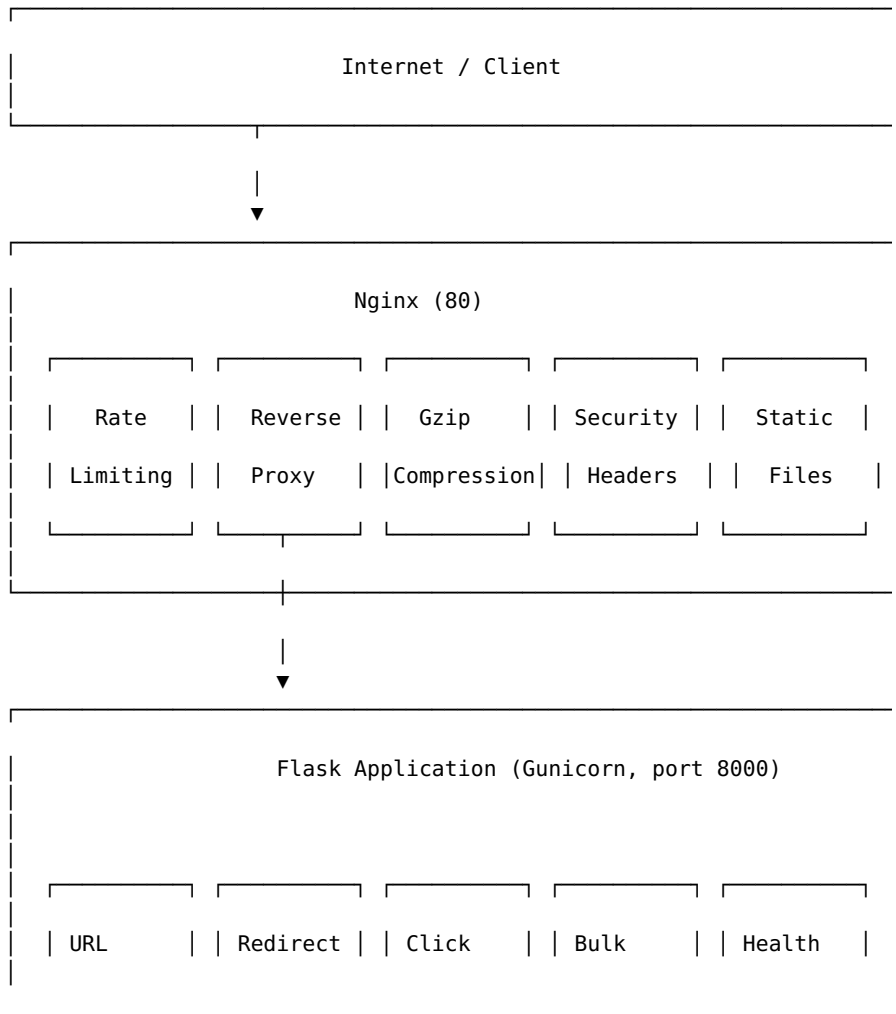
Scalable URL Shortener - Documentation

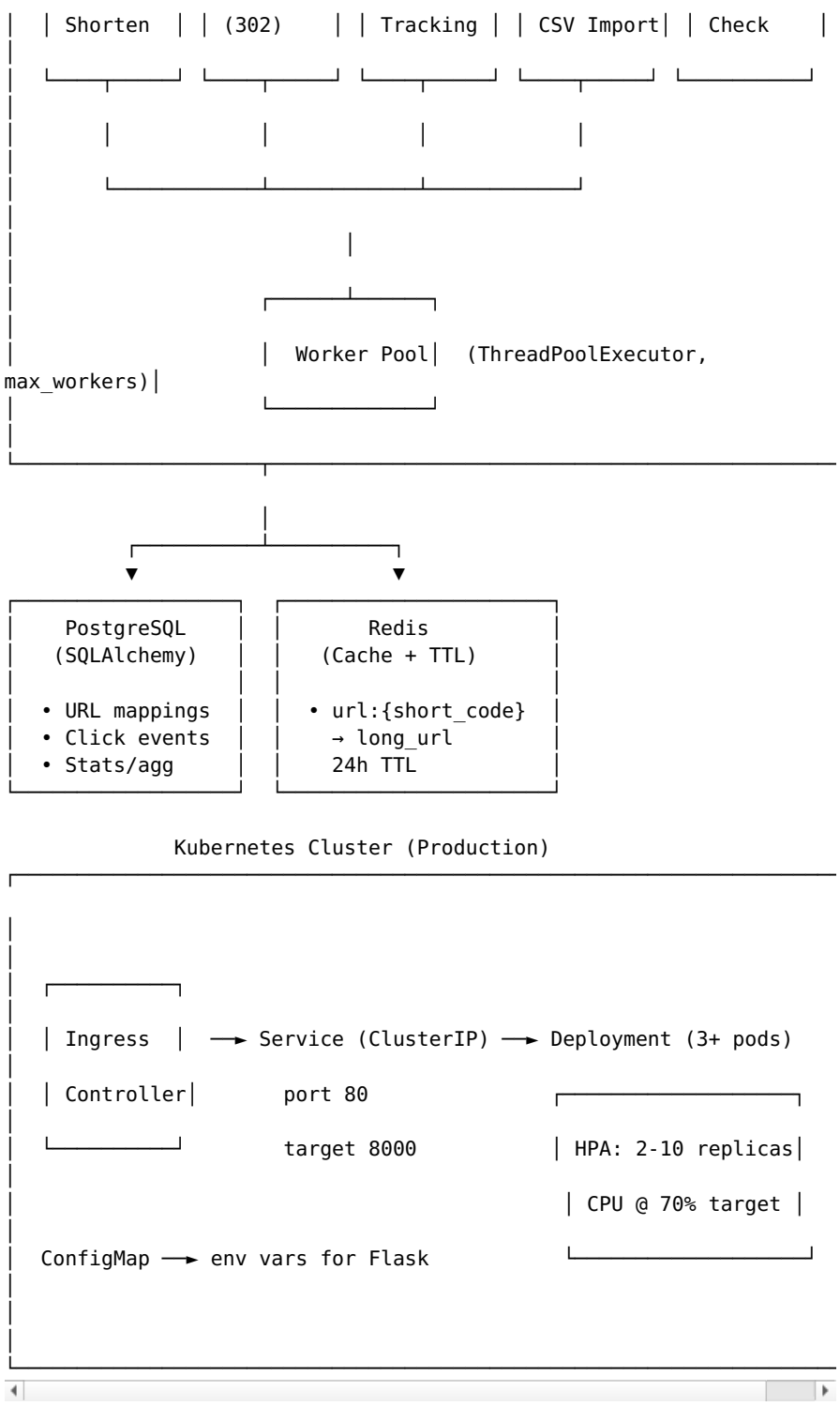
Scalable URL Shortener with Distributed Orchestration

A production-grade URL shortening service built with Python/Flask, featuring distributed orchestration via Docker Compose and Kubernetes. Designed to demonstrate virtualization concepts including containerization, horizontal pod autoscaling, reverse proxy-based rate limiting, caching strategies, and CI/CD automation — ideal for a university Virtualization Systems course.

The service accepts long URLs via REST API or web UI, generates unique short codes, and provides fast redirects backed by an in-memory Redis cache. Click tracking, bulk CSV import with concurrent worker pools, and a statistics API are built in. Nginx provides rate limiting, reverse proxying, and security hardening.

Architecture





Features

- **URL Shortening** — POST a long URL and receive a unique 6-character short code. Duplicate URLs are detected and return the existing short code.
- **Redirect with Click Tracking** — Each redirect (HTTP 302) records the timestamp, User-Agent, referrer, and client IP for analytics.
- **Bulk CSV Import with Worker Pools** — Upload a CSV or plain-text file of URLs. Concurrent workers (`ThreadPoolExecutor`) validate, deduplicate, and batch-insert into PostgreSQL. Results are cached to Redis in a single pipeline.

- **Redis Caching** — Fast lookup layer with 24-hour TTL. Cache-aside pattern: check Redis first, fall back to PostgreSQL, then populate cache.
- **Rate Limiting** — Two layers: Nginx `limit_req` (30 req/s per IP) + application-level sliding-window rate limiter in Flask (per-client configurable limit).
- **Statistics API** — Retrieve total clicks, recent clicks (last 24h), and creation timestamp for any short code.
- **Health Probes** — Kubernetes-ready `/api/health` endpoint that checks database connectivity and Redis availability.
- **Docker & Docker Compose** — Multi-service orchestration with health checks, resource constraints, and dependency ordering.
- **Kubernetes Deployment** — Namespace, ConfigMap, Deployment, Service (ClusterIP), HorizontalPodAutoscaler, and Ingress manifests included.
- **CI/CD Pipeline** — GitHub Actions runs tests, builds/pushes Docker images, and deploys to a VPS via SSH.

Tech Stack

Component	Technology
Application	Python 3.11, Flask 3.x, Gunicorn
Database	PostgreSQL 16 (Alpine), SQLAlchemy ORM
Cache	Redis 7 (Alpine)
Reverse Proxy	Nginx 1.25 (Alpine)
Containerization	Docker, Docker Compose
Orchestration	Kubernetes (K3s / Minikube)
CI/CD	GitHub Actions
Testing	Pytest
CORS	Flask-CORS

Quick Start — Docker Compose

Prerequisites

- Docker Engine 24+ and Docker Compose v2
- `curl` (for health check verification)
- Git

Steps

```
# 1. Clone the repository
git clone https://github.com/aminghuf/URL_shortner.git
cd URL_shortner

# 2. (Optional) Customize environment variables
export POSTGRES_DB=urlshortener
export POSTGRES_USER=urlshortener
export POSTGRES_PASSWORD=urlshortener_secret
export BULK_IMPORT_WORKERS=4

# 3. Build and start all services
docker compose up --build -d

# 4. Verify health
curl -fs http://localhost:80/api/health | python3 -m json.tool
```

```bash
# 5. Shorten a URL
curl -X POST http://localhost:80/shorten \
  -H "Content-Type: application/json" \
  -d '{"url": "https://example.com"}'

# 6. Test redirect
curl -v http://localhost:80/AbCdEf

# 7. View stats
curl http://localhost:80/stats/AbCdEf

# 8. Tear down
docker compose down -v
```

Default Services

Service	Port(s)	Purpose
`nginx`	`80`	Reverse proxy, rate limiting, static
`app`	`8000` (internal)	Flask application (Gunicorn)
`postgres`	`5432` (internal)	Primary database
`redis`	`6379` (internal)	Cache layer

API Documentation

All endpoints are served at `http://localhost:{port}` (port `80` through Nginx, or `8000` directly).

Endpoints

Method	Path	Description	Request Body / Query
`GET`	`/`	Render the landing page (web UI)	—
`GET`	`/health`	Legacy health check	—
`GET`	`/api/health`	Dependency-aware health probe (DB + Redis checks)	—
`POST`	`/shorten`	Create a short URL	`{"url": "https://example.com"}` (JSON)
`GET`	`/<short_code>`	Redirect to the original long URL (with click tracking)	—
`GET`	`/stats/<short_code>`	Get click statistics for a short code	—

`POST`	`/bulk-import`	Bulk import URLs from a CSV or plain-text file	Multipart form: `file=<csv_or_txt>`

Error Responses

Status	Meaning	Body
`400`	Bad Request	`{"error": "URL is required"}`
`404`	Not Found	`{"error": "URL not found"}`
`429`	Too Many Requests	`{"error": "Rate limit exceeded. Try again later."}`
`500`	Internal Server Error	`{"error": "Database insert failed", "detail": "…"}`
`503`	Service Unavailable	`{"status": "degraded", "database": "down", …}`

Kubernetes Deployment

A complete set of Kubernetes manifests is located in the `k8s/` directory. Deploy in order below.

1. Namespace

```
kubectl apply -f k8s/namespace.yaml
```

```yaml
# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: url-shortener
  labels:
    app: url-shortener
    environment: production
```

2. ConfigMap

```
kubectl apply -f k8s/configmap.yaml
```

```yaml
# k8s/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: url-shortener-config
  namespace: url-shortener
  labels:
    app: url-shortener
data:
  DATABASE_URL: "postgresql://urlshortener:***@postgres-service:5432/urlshortener"
  REDIS_URL: "redis://redis-service:6379/0"
  APP_NAME: "url-shortener"
  FLASK_ENV: "production"
```

> **Note:** The PostgreSQL password and other secrets should be stored in a Kubernetes `Secret` (not `ConfigMap`) in production. The manifests here use a ConfigMap with a placeholder password for demonstration.

3. Deployment

```
kubectl apply -f k8s/deployment.yaml
```

```yaml
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: url-shortener
  namespace: url-shortener
  labels:
    app: url-shortener
spec:
  replicas: 3
  selector:
    matchLabels:
      app: url-shortener
  template:
    metadata:
      labels:
        app: url-shortener
    spec:
      containers:
        - name: url-shortener
          image: url-shortener:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
              name: http
          envFrom:
            - configMapRef:
                name: url-shortener-config
          resources:
            requests:
              cpu: "200m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
          livenessProbe:
            httpGet:
              path: /api/health
              port: 8000
            initialDelaySeconds: 10
            periodSeconds: 15
            timeoutSeconds: 5
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /api/health
              port: 8000
            initialDelaySeconds: 5
            periodSeconds: 10
            timeoutSeconds: 3
            failureThreshold: 2
          startupProbe:
            httpGet:
              path: /api/health
              port: 8000
            initialDelaySeconds: 3
```

```
            periodSeconds: 5
            failureThreshold: 30
```

4. Service

```
kubectl apply -f k8s/service.yaml
```

```
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
  name: url-shortener-service
  namespace: url-shortener
  labels:
    app: url-shortener
spec:
  type: ClusterIP
  selector:
    app: url-shortener
  ports:
    - name: http
      port: 80
      targetPort: 8000
      protocol: TCP
```

5. HorizontalPodAutoscaler

```
kubectl apply -f k8s/hpa.yaml
```

```
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: url-shortener-hpa
  namespace: url-shortener
  labels:
    app: url-shortener
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: url-shortener
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

6. Ingress

```
kubectl apply -f k8s/ingress.yaml
```

```
# k8s/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: url-shortener-ingress
  namespace: url-shortener
  labels:
    app: url-shortener
  annotations:
```

```
    nginx.ingress.kubernetes.io/rewrite-target: /
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - host: url-shortener.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: url-shortener-service
                port:
                  number: 80
```

Full Deployment Script

```
# One-shot deployment
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/hpa.yaml
kubectl apply -f k8s/ingress.yaml

# Verify
kubectl -n url-shortener get all
kubectl -n url-shortener get hpa
```

Infrastructure Dependencies

You must also deploy PostgreSQL and Redis in the cluster (or use managed services). Minimal example:

```
# PostgreSQL
kubectl -n url-shortener run postgres --image=postgres:16-alpine \
  --env="POSTGRES_DB=urlshortener" \
  --env="POSTGRES_USER=urlshortener" \
  --env="POSTGRES_PASSWORD=urlshortener_secret" \
  --port=5432
kubectl -n url-shortener expose pod postgres --name=postgres-service
--port=5432

# Redis
kubectl -n url-shortener run redis --image=redis:7-alpine --
port=6379
kubectl -n url-shortener expose pod redis --name=redis-service --
port=6379
```

Environment Variables

Variable	Required	Default	Descri
`DATABASE_URL`	Yes	`sqlite:///urls.db`	PostgreSC connectio (auto-con `postgres:` `postgresq`
`REDIS_URL`	No	`redis://localhost:6379/0`	Redis con string
			Max threa the

`BULK_IMPORT_WORKERS`	No	`4`	`ThreadPoo` for bulk i
`FLASK_ENV`	No	`production`	Flask env mode
`POSTGRES_DB`	Yes*	`urlshortener`	Database (for `docke` `compose u`
`POSTGRES_USER`	Yes*	`urlshortener`	Database `docker co`
`POSTGRES_PASSWORD`	Yes*	`urlshortener_secret`	Database password `docker co`

> * Required only when using Docker Compose; in Kubernetes, these are passed via ConfigMap/Secret.

Development Setup

Prerequisites

- Python 3.11+
- pip (or uv)
- PostgreSQL 16 (or SQLite for local dev)
- Redis 7 (optional — cache degrades gracefully)
- Docker (optional — for containerized dev)

Local Without Docker

```
        # 1. Clone and enter repo
        git clone https://github.com/aminghuf/URL_shortner.git
        cd URL_shortner

        # 2. Create virtual environment
        python3 -m venv venv
        source venv/bin/activate

        # 3. Install dependencies
        pip install -r requirements.txt

        # 4. (Optional) Install dev extras
        pip install pytest

        # 5. Set up environment
        export DATABASE_URL="sqlite:///dev.db"          # Use SQLite for
local dev
        export REDIS_URL="redis://localhost:6379/0"     # Or omit to skip
Redis
        export FLASK_ENV="development"
        export BULK_IMPORT_WORKERS="2"

        # 6. Run the application
        flask run --debug --port 8000
```

With Docker (Development Overrides)

```
        # Override the compose file for hot-reload
        docker compose -f docker-compose.yml up --build -d
        docker compose logs -f app
```

Testing

Running Tests

```
# With virtual environment active
pytest -v

# With coverage (install pytest-cov first)
pytest --cov=app --cov-report=term-missing

# Run a specific test
pytest tests/test_app.py::test_shorten_url -v
```

Test Suite Overview

The test suite (`tests/test_app.py`) covers:

Test	Assertions
`test_health`	`GET /health` returns `200 {"status": "ok"}`
`test_shorten_url`	`POST /shorten` with valid URL returns `201` + short code
`test_shorten_missing_url`	`POST /shorten` without URL returns `400`
`test_redirect_to_url`	`GET /<short_code>` returns `302` with correct `Location` header

To extend: run `pytest` after adding routes, and verify no regressions.

CI/CD Pipeline

The project uses **GitHub Actions** (`.github/workflows/test.yml`) with two jobs:

`test` Job

Triggered on every push to `main` and on pull requests.

1. **Checkout** — `actions/checkout@v4`
2. **Set up Python** — `actions/setup-python@v5` (Python 3.11)
3. **Install dependencies** — `pip install -r requirements.txt`
4. **Run tests** — `pytest`
5. **Build Docker image** — `docker build -t url-shortener:test .`
6. **Login to Docker Hub** — authenticates with `${{ secrets.DOCKER_USERNAME }}` and `${{ secrets.DOCKER_PASSWORD }}`
7. **Tag & Push** — pushes `:latest` and `:<commit-sha>` tags to Docker Hub

`deploy` Job

Runs after `test` completes successfully.

1. **SSH into VPS** — uses `appleboy/ssh-action@v1.0.3` with `VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY` secrets
2. **Stop & remove** existing container
3. **Pull** latest image from Docker Hub
4. **Run** new container with Docker on port 5000
5. **Health check** — verifies the deployment with `curl -f http://127.0.0.1:5000/health`

Resource Constraints & Scaling

Docker Compose Resource Limits

Service	CPU Limit	Memory Limit	CPU Reservation	Memory Reservation
`app`	0.5 CPU	512 MB	0.2 CPU	256 MB
`nginx`	0.2 CPU	128 MB	0.1 CPU	64 MB
`postgres`	0.5 CPU	512 MB	0.2 CPU	256 MB
`redis`	0.3 CPU	256 MB	0.1 CPU	128 MB

Kubernetes Resource Requests & Limits

Resource	Request	Limit	Rationale
CPU	200m	500m	200m guarantees baseline; 500m burst
Memory	256 MiB	512 MiB	Flask + Gunicorn + SQLAlchemy overhead

Horizontal Pod Autoscaler (HPA)

- **Minimum replicas:** 2 (high availability)
- **Maximum replicas:** 10 (burst capacity)
- **Metric:** CPU utilization at **70%** target
- **Cooldown/Stabilization:** Kubernetes default (5 min scale-up, 3 min scale-down)

Nginx Rate Limiting

- **Application layer:** Sliding-window counter in Flask (resets per minute, uses Redis or in-memory dictionary). Configured per-client IP.
- **Proxy layer (Nginx):** `limit_req_zone` with 10 MB shared memory zone (~160,000 IPs tracked), **30 requests/second** per IP, burst of 20, and nodelay.
- **Connection limiting:** Nginx `limit_conn` at 10 concurrent connections per IP.

Performance Characteristics

Scenario	Expected Throughput (Docker)	With HPA (K8s, 5 replicas)
Redirect (cache hit)	~5,000 req/s	~20,000 req/s
Shorten (DB write)	~500 req/s	~2,000 req/s
Bulk import (1000 URLs)	~5–10 seconds	~2–4 seconds
Stats (cache + DB read)	~3,000 req/s	~12,000 req/s

> **Note:** Throughput figures are estimates based on local testing with a 4-core / 8 GB host. Actual performance depends on network latency, disk I/O, and Redis/PostgreSQL tuning.

Project Structure

```
URL_shortner/
├── app.py                      # Flask application (routes, models,
caching, rate limiting)
├── requirements.txt            # Python dependencies
├── Dockerfile                  # Multi-stage Python container
├── docker-compose.yml          # Multi-service orchestration (4
containers)
├── .gitignore
│
├── k8s/                        # Kubernetes manifests
│   ├── namespace.yaml          # Isolated namespace
│   ├── configmap.yaml          # Environment configuration
│   ├── deployment.yaml         # App deployment (3 replicas,
probes, resources)
│   ├── service.yaml            # ClusterIP service (port 80 → 8000)
│   ├── hpa.yaml                # Horizontal pod autoscaler (2–10
replicas)
│   └── ingress.yaml            # Nginx ingress controller rules
│
├── nginx/                      # Nginx reverse proxy
│   ├── Dockerfile              # Alpine-based nginx image
│   └── nginx.conf              # Rate limiting, reverse proxy,
security headers
│
├── templates/
│   └── index.html              # Landing page (web UI)
│
├── static/
│   └── style.css               # Frontend styles
│
├── tests/
│   └── test_app.py             # Pytest test suite
│
└── .github/workflows/
    └── test.yml                # GitHub Actions CI/CD pipeline
```

Course Reflection — Virtualization Systems

This project demonstrates several core virtualization concepts taught in a university Virtualization Systems course:

1. **Containerization** — Each service (app, nginx, postgres, redis) runs in its own Docker container with isolated filesystem, network, and process space. The `Dockerfile` uses a slim base image (`python:3.11-slim` → `307 MB`) and runs as a non-root user for security.

2. **Multi-service Orchestration** — Docker Compose orchestrates four interdependent containers with health checks, dependency ordering (`depends_on` with `condition: service_healthy`), and resource constraints.

3. **Kubernetes Orchestration** — Full set of manifests demonstrating namespaces, ConfigMaps, Deployments with rolling updates, ClusterIP Services, HorizontalPodAutoscaler for elastic scaling, and Ingress for external traffic routing.

4. **Reverse Proxy Patterns** — Nginx acts as a secure entry point, demonstrating rate limiting, connection limiting, request buffering, and security header injection — a common pattern in virtualized microservice architectures.

5. **Horizontal Scaling** — The HPA automatically provisions additional pods when CPU exceeds 70%, demonstrating elastic horizontal scaling in response to load.

6. **Health Probes** — Liveness, readiness, and startup probes ensure the Kubernetes scheduler only routes traffic to healthy pods and automatically restarts failed containers.

7. **CI/CD Automation** — GitHub Actions automates testing, container image building, and deployment — demonstrating the virtualization development lifecycle.

License

Authors

- **amin** — @aminghuf
- **shakibofski** — Frontend development